

Bike Demand Prediction - Results

Random Forest regression was used to predict hourly bike rental demand. The data was kept in chronological order and split into 75% training and 25% validation data.

Model evaluation

min_samples_leaf	Validation MAE
1	46.04
2	46.30
5	47.87
10	51.30

The lowest validation MAE was **46.04 rentals** with **min_samples_leaf = 1**, so that setting was used for the final model.

Training and model selection

Four Random Forest settings were compared using Mean Absolute Error (MAE).

```
:22]
✓ 0s df["dteday"]=pd.to_datetime(df["dteday"])
      df=df.sort_values(["dteday", 'hr'])
```

```
:23]
✓ 0s y=df.cnt
      features=['mnth', 'yr', 'hr', 'weekday', 'holiday', 'weathersit', 'hum', 'windspeed', 'temp', 'workingday']
      X=df[features]
```

```
:35]
✓ 0s train_X, val_X, train_y, val_y=train_test_split(X, y, test_size=0.25, shuffle=False)
```

```
:41]
✓ 16s  for leaves in [1,2,5,10]:
      model=RandomForestRegressor(min_samples_leaf=leaves, random_state=1)
      model.fit(train_X, train_y)
      modelPredict=model.predict(val_X)
      mae=mean_absolute_error(val_y, modelPredict)
      print(f"Leaves: {leaves}, MAE: {mae:.2f}")
```

▼ ... Leaves: 1, MAE: 46.04
Leaves: 2, MAE: 46.30
Leaves: 5, MAE: 47.87
Leaves: 10, MAE: 51.30

Actual vs. predicted values

The model's predictions were placed beside the actual validation values for direct comparison.

```
 comparison=pd.DataFrame({  
    "Actual":val_y,  
    "Predicted":model1Predict  
})  
  
comparison.head(10)
```

...

Actual Predicted



13034 398 468.64

13035 568 616.60

13036 295 271.76

13037 174 174.56

13038 201 191.86

13039 257 268.51

13040 236 217.12

13041 230 219.23

13042 234 257.38

Error analysis by hour

Average absolute error was calculated for each hour. The highest errors occurred at 17:00, 18:00, 08:00, 19:00, and 16:00.

```
comparison["Absolute_Error"]=(
    comparison["Actual"]-comparison["Predicted"]
).abs()

comparison["hr"]=val_X["hr"]

error_by_hour=(
    comparison.groupby("hr")["Absolute_Error"].mean().sort_values(ascending=False)
)

print(error_by_hour.head(5))
```

```
hr
17    110.743352
18     99.776538
8      92.580157
19     87.725330
16     80.456978
Name: Absolute_Error, dtype: float64
```

Hourly error summary

The hourly summary compares mean actual demand, predicted demand, and absolute error.

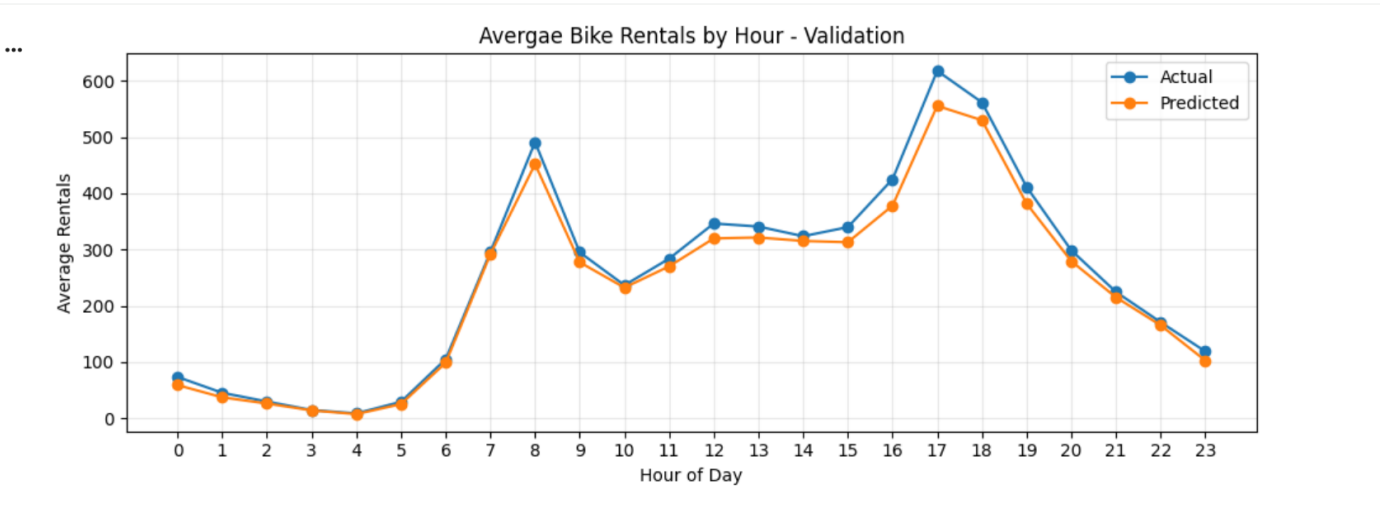
```
hour_summary=comparison.groupby("hr")["Actual", "Predicted", "Absolute_Error"].mean()

print(hour_summary.sort_values("Absolute_Error", ascending=False).head(5).round(2))
```

	Actual	Predicted	Absolute_Error
hr			
17	618.08	555.76	110.74
18	562.01	530.21	99.78
8	491.34	452.14	92.58
19	411.56	381.97	87.73
16	425.31	378.26	80.46

Average hourly demand: actual vs. predicted

The predicted values follow the overall daily demand pattern, including the morning and evening peaks. The largest gaps appear during high-demand commuting hours.



Data quality checks

The dataset contained no missing values, duplicate rows, or duplicate date-hour pairs.

```
... Missing Values:
instant      0
dteday       0
season       0
yr           0
mnth         0
hr           0
holiday      0
weekday      0
workingday   0
weathersit    0
temp         0
atemp        0
hum          0
windspeed    0
casual       0
registered   0
cnt          0
dtype: int64
```

Duplicate rows: 0

Duplicate date-hour pairs: 0

Conclusion

What the model does

The model uses month, year, hour, weekday, holiday status, weather situation, humidity, wind speed, temperature, and working-day status to estimate the number of bike rentals for an hour. Random Forest builds multiple decision trees and combines their predictions to produce the final estimate.

What we found

The best tested setting achieved a validation MAE of 46.04 rentals. The model follows the main hourly demand pattern and captures both the morning and evening peaks, although prediction errors become larger during the busiest hours.

What this project demonstrates

The project covers a complete introductory machine-learning workflow: loading and checking data, keeping time order during the train/validation split, selecting features, training and comparing model settings, evaluating predictions with MAE, comparing actual and predicted values, and analysing where the model makes its largest errors.

Possible next step

Future work could test additional features, broader hyperparameter tuning, time-series cross-validation, or other regression models to improve performance during peak-demand periods.